

Transformer Computational Theory

English seminar study notes

These notes are an English companion to the seminar handout. The emphasis is on mathematical derivations that can be followed line by line, on separating theorem statements from interpretation, and on identifying the assumptions under which each claim is valid.

Primary readings

Hahn, **Theoretical Limitations of Self-Attention in Neural Sequence Models**, TACL 2020.

Pérez et al., **Attention is Turing Complete**, JMLR 2021.

Merrill and Sabharwal, **The Parallelism Tradeoff**, TACL 2023.

Contents

1	Notation and the resource model	3
1.1	Four axes that must be separated	3
1.2	A map of the main results	3
2	Formal languages and circuit complexity	4
2.1	Language recognition	4
2.2	AC^0 , TC^0 , and NC^1	4
2.3	Regular languages and sequential state	4
3	Fixed-depth self-attention and early limitations	5
3.1	Why global attention is not automatically sequential computation	5
3.2	Hahn’s perspective	5
3.3	Hard attention as a circuit	5
4	Log-precision Transformers and TC^0	6
4.1	Why precision must be part of the theorem	6
4.2	The upper-bound idea	6
4.3	Why attention sums do not imply serial addition	6
4.4	What the upper bound means	6
5	Why Turing completeness is not a contradiction	7
5.1	Turing completeness	7
5.2	Pérez–Barceló–Marinković construction	7
5.3	Reading a simulated tape	7
5.4	Where the extra power comes from	7
6	Chain of Thought as a computational resource	8
6.1	One generated token is another round of computation	8
6.2	A time–space sandwich	8
6.3	Regular languages with linear scratchpad	8
6.4	Polynomial scratchpad and polynomial-time computation	8
7	Parallel shortcuts and automata	9
7.1	Balanced composition of transitions	9
7.2	Why this matters for Transformers	9
7.3	Existence versus learnability	9
8	Arithmetic and algorithmic reasoning	10
8.1	TC^0 already contains substantial arithmetic	10
8.2	Sequential dependency is often the harder resource	10
8.3	Internal versus external reasoning	10
9	Presentation checklist	11
9.1	Ten formulas and relations to keep	11
9.2	Common mistakes	11

Bibliography**12**

1

Notation and the resource model

1.1 Four axes that must be separated

Transformer computational theory looks contradictory only when different resource models are mixed. Keep separate:

1. architectural depth L ;
2. sequence length n ;
3. numerical precision p ;
4. number of autoregressive decoding steps T .

A fixed-depth, log-precision one-pass Transformer and an autoregressive Transformer with an unbounded scratchpad are different computational objects.

1.2 A map of the main results

Result type	Main assumption	Message
circuit upper bound	fixed depth, bounded/log precision	one pass lies in a small parallel circuit class
Turing completeness	idealized precision/state and recurrent decoding	arbitrary computable procedures can be simulated
CoT hierarchy	repeated autoregressive steps	scratchpad length is an explicit time resource
formal-language results	task-specific assumptions	reveal which sequential dependencies require extra depth/time

How to say it in the presentation The right question is never simply “Are Transformers powerful?” It is: powerful under which depth, precision, sequence-length, and decoding-time budget?

2

Formal languages and circuit complexity

2.1 Language recognition

A language is a set $L \subseteq \Sigma^*$. For each length n , recognition asks for a Boolean function

$$f_n : \Sigma^n \rightarrow \{0, 1\}.$$

A circuit family $\{C_n\}$ contains one circuit for every input length. Complexity classes restrict depth, size, gate types, and uniformity of the family.

2.2 AC^0 , TC^0 , and NC^1

At a high level:

- AC^0 : polynomial size, constant depth, unbounded fan-in AND/OR/NOT;
- TC^0 : adds threshold/majority gates;
- NC^1 : polynomial size and $O(\log n)$ depth with bounded fan-in.

Threshold gates make TC^0 surprisingly strong: many arithmetic operations, including addition and multiplication under standard encodings, have highly parallel circuits.

2.3 Regular languages and sequential state

A DFA has transition function

$$\delta : Q \times \Sigma \rightarrow Q.$$

For a string $x_1 \cdots x_n$, the final state is

$$q_n = \delta(\cdots \delta(\delta(q_0, x_1), x_2) \cdots, x_n).$$

The definition is sequential, but this does not imply that every DFA evaluation inherently requires depth n : transition functions can sometimes be composed in a balanced tree.

How to say it in the presentation Formal-language difficulty and circuit depth are related but not identical. A sequential specification can sometimes admit a parallel algorithm.

3

Fixed-depth self-attention and early limitations

3.1 Why global attention is not automatically sequential computation

Self-attention can read all positions in one layer, but a constant number of layers provides only a constant number of rounds of adaptive computation. Global access is therefore different from an unbounded recurrent state update.

3.2 Hahn's perspective

Early lower-bound analyses show that under specific attention assumptions, fixed-depth self-attention has difficulty with language phenomena requiring certain forms of unbounded hierarchical or counting behavior. The technical statement depends on the attention type and numerical assumptions; one should not paraphrase it as “Transformers cannot recognize regular languages.”

3.3 Hard attention as a circuit

With hard attention, every head selects a bounded amount of information from previous representations. A fixed number of layers can often be unfolded into a bounded-depth circuit. This makes the architectural depth constraint transparent.

How to say it in the presentation A single attention layer has global communication but only one round of computation. Computational depth, not receptive field alone, is the key resource.

4

Log-precision Transformers and TC^0

4.1 Why precision must be part of the theorem

A real number with arbitrary precision can hide unbounded information. To obtain a meaningful discrete complexity statement, numerical values must have bounded bit complexity, for example $O(\log n)$ bits.

4.2 The upper-bound idea

For fixed architectural depth and log-bounded precision, the elementary operations in a Transformer layer—dot products, bounded-precision arithmetic, comparisons/softmax approximations, normalization, and feed-forward maps—can be implemented by constant-depth threshold circuits of polynomial size. Composing a constant number of layers preserves constant depth, leading to a uniform TC^0 -type upper bound under the theorem’s assumptions.

4.3 Why attention sums do not imply serial addition

An attention output contains sums such as

$$\sum_{j=1}^n \alpha_{ij} v_j.$$

Circuit complexity allows these sums to be computed in parallel with threshold/arithmetic circuitry. The mathematical expression has n terms, but circuit depth is not the same as the number of written summands.

4.4 What the upper bound means

The result constrains one-pass computation with fixed depth and prescribed precision. It does not say that a Transformer cannot solve tasks outside TC^0 when given more layers, iterative decoding, external memory, or a different precision model.

How to say it in the presentation The TC^0 result is a resource-bounded simulation theorem: a fixed-depth, log-precision forward pass can be compiled into a constant-depth threshold circuit.

5

Why Turing completeness is not a contradiction

5.1 Turing completeness

A model is Turing complete if, under an appropriate encoding and with sufficient time/memory, it can simulate any Turing machine. This is an asymptotic expressivity statement, not an efficiency guarantee.

5.2 Pérez–Barceló–Marinković construction

Idealized Transformer constructions can encode a machine configuration into activations, use attention to retrieve relevant prior information, and generate the next configuration autoregressively. Positional information and sufficiently precise numerical state are crucial to the simulation.

5.3 Reading a simulated tape

A common device is to encode write events over time. To recover the content of a tape cell, the model identifies the most recent write to that address. Attention supplies content-addressable retrieval, while the autoregressive loop supplies repeated machine steps.

5.4 Where the extra power comes from

The one-pass TC^0 theorem fixes depth. A Turing simulation uses an unbounded number of recurrent or autoregressive steps, and often stronger precision assumptions. Therefore the two results answer different questions.

Caution Turing complete does not mean polynomial-time efficient, numerically robust, trainable by SGD, or capable of generalizing to arbitrary computation lengths from finite data.

How to say it in the presentation There is no paradox: one fixed forward pass can lie in a low circuit class while repeated forward passes, with generated state fed back as input, simulate an arbitrary algorithm.

6

Chain of Thought as a computational resource

6.1 One generated token is another round of computation

If a decoder produces scratchpad tokens

$$z_1, z_2, \dots, z_T,$$

then each token invokes another Transformer forward pass conditioned on the previous transcript. The effective sequential depth therefore grows with T even when the network parameters and architectural depth are fixed.

6.2 A time–space sandwich

Results in this line of work relate a Transformer with $t(n)$ generated reasoning steps to classical time and space. A useful schematic sandwich is

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n)) \subseteq \text{SPACE}(t(n) + \log n),$$

with the exact model definitions and uniformity assumptions taken from the corresponding theorem. The left inclusion comes from simulating algorithmic steps with tokens; the right inclusion comes from storing the transcript and recomputing each bounded-depth forward pass with limited workspace.

6.3 Regular languages with linear scratchpad

A finite automaton can be simulated by emitting its current state after reading each input symbol. If $q_i = \delta(q_{i-1}, x_i)$, a scratchpad of length $\Theta(n)$ explicitly externalizes the recurrent state sequence.

6.4 Polynomial scratchpad and polynomial-time computation

If a polynomial-time Turing machine takes $p(n)$ steps, an autoregressive model that can faithfully emit one simulated configuration per step uses $p(n)$ reasoning tokens. Thus polynomially many CoT steps can recover polynomial-time sequential computation under the idealized simulation assumptions.

How to say it in the presentation CoT is not merely “explanation text” in these theorems. It is an external memory and time tape that converts repeated constant-depth computations into a long sequential algorithm.

7

Parallel shortcuts and automata

7.1 Balanced composition of transitions

For each symbol a , define the state transition map $T_a : Q \rightarrow Q$. A substring $x_i \cdots x_j$ corresponds to the composition

$$T_{x_j} \circ \cdots \circ T_{x_i}.$$

Function composition is associative, so a length- n product of transition maps can be organized as a balanced tree of depth $O(\log n)$ rather than a left-to-right chain of depth n .

7.2 Why this matters for Transformers

Some formal-language tasks that look sequential from the automaton definition admit logarithmic-depth parallel algorithms. Results constructing Transformer shortcuts exploit this algebraic structure. Their existence does not imply that standard training automatically discovers the shortcut.

7.3 Existence versus learnability

The theoretical statements often show that parameters exist that implement a circuit or simulation. A separate theorem would be required to prove that gradient-based training on a specified data distribution finds those parameters.

How to say it in the presentation Sequential specification does not equal sequential lower bound. Before claiming that CoT is necessary, ask whether the transition computation admits an associative parallelization.

8

Arithmetic and algorithmic reasoning

8.1 TC^0 already contains substantial arithmetic

The fact that a one-pass model is upper-bounded by TC^0 does not make it arithmetically trivial. Threshold circuits can implement many operations that look complicated when written sequentially.

8.2 Sequential dependency is often the harder resource

Long division-like procedures, graph exploration, or iterative dynamic programs are difficult not merely because they contain arithmetic, but because later operations depend adaptively on earlier intermediate states. Autoregressive reasoning provides the missing recurrence by externalizing those states.

8.3 Internal versus external reasoning

A fixed forward pass can still perform rich parallel transformations internally. The theoretical distinction is that hidden activations are consumed in a bounded number of layers, while externalized tokens can be fed back for an unbounded number of additional rounds.

How to say it in the presentation The strongest conceptual distinction is parallel depth versus sequential time. Transformers have strong global parallel computation; CoT supplies additional sequential time.

9

Presentation checklist

9.1 Ten formulas and relations to keep

$L \subseteq \Sigma^*$,
 $q_i = \delta(q_{i-1}, x_i)$,
 $AC^0 \subseteq TC^0 \subseteq NC^1$ (standard containments),
 fixed depth + log precision $\Rightarrow$ TC^0 -type simulation,
 Turing completeness $\neq$ efficient computation,
 $z_1, \dots, z_T$ adds T sequential rounds,
 $TIME(t) \subseteq CoT(t)$,
 $CoT(t) \subseteq SPACE(t + \log n)$,
 $T_{x_n} \circ \dots \circ T_{x_1}$,
 parallel depth $\neq$ sequence length.

9.2 Common mistakes

- Calling a Turing-complete model efficient or learnable.
- Stating a TC^0 upper bound without its fixed-depth and precision assumptions.
- Treating global attention as equivalent to unbounded sequential computation.
- Assuming every apparently sequential problem needs linear CoT .
- Confusing existence of a simulation with empirical behavior of trained LLMs.

How to say it in the presentation Thirty-second conclusion: a fixed-depth, finite-precision Transformer forward pass is highly parallel and can be upper-bounded by small circuit classes, while autoregressive reuse of the same network adds sequential time and can support general algorithm simulation. Precision and scratchpad length are computational resources, not implementation details.

Bibliography

- Hahn, M. *Theoretical Limitations of Self-Attention in Neural Sequence Models*. TACL, 2020.
- Pérez, J., Barceló, P., and Marinković, J. *Attention is Turing Complete*. JMLR, 2021.
- Merrill, W. and Sabharwal, A. *The Parallelism Tradeoff: Limitations of Log-Precision Transformers*. TACL, 2023.
- Merrill, W. and Sabharwal, A. *The Expressive Power of Transformers with Chain of Thought*. 2024.